Python & Algorithms for Coding Interviews

Section 1 — Python for Algorithmic Thinking

1.1 Python Execution Model

1.1.1 How Python Runs Your Code

Most people think Python just 'reads and runs' code line by line. That mental model is incomplete — and it causes confusion when debugging recursive functions, understanding scope, or analysing performance.

Here is what actually happens when you run a Python file.

Step 1 — Source Code: You write solution.py. This is plain text.

Step 2 — Compilation to Bytecode: Python compiles source code into bytecode — a lower-level, platform-independent instruction set. Stored in .pyc files inside __pycache__.

Step 3 — Execution by the PVM: The Python Virtual Machine (PVM) reads the bytecode and executes it instruction by instruction.

```
Your Code (.py)
      ↓
  Compiler
      ↓
 Bytecode (.pyc)
      ↓
Python Virtual Machine (PVM)
      ↓
   Output
```

Interview	When asked "Is Python interpreted or compiled?" the correct answer is both. It compiles to bytecode first, then the PVM interprets that bytecode. This detail impresses interviewers.

1.1.2 The Interpreter and Bytecode

The default Python implementation is CPython — written in C. When you type python in your terminal, you are running CPython. You can inspect the bytecode Python generates using the dis module.

```
import dis

def add(a, b):
    return a + b

dis.dis(add)

# Output:
# LOAD_FAST   a
# LOAD_FAST   b
# BINARY_ADD
# RETURN_VALUE
```

Each line is a bytecode instruction. The PVM executes these one by one. Python does not compile to machine code directly — every bytecode instruction goes through the PVM. This is one reason Python is slower than compiled languages, and why algorithm efficiency matters more in Python.

Terms	CPython · bytecode · PVM · interpreted language

1.1.3 The Call Stack

This is one of the most important mental models for understanding recursion, function calls, and stack overflow errors. Every time you call a function, Python creates a stack frame and pushes it onto the call stack. When the function returns, the frame is popped off.

```
def c():
    return 1

def b():
    return c()

def a():
    return b()

a()

# Call stack during execution:
```

```
# |   c()    |  ← top (currently executing)
# |   b()    |
# |   a()    |
# | main()   |  ← bottom
```

Each frame stores: local variables · return address · function arguments

Recursion limit: Python's default is 1000 frames. Hitting it means you need an iterative solution with an explicit stack — always safer in interviews.

```
import sys
sys.setrecursionlimit(10000)  # increase if needed
```

1.1.4 Stack vs Heap Memory

Memory Region	Stores	Size	Speed
Stack	Function frames, local variable references	Small, fixed	Very fast
Heap	Actual objects (lists, dicts, instances)	Large, dynamic	Slower

Key insight: Variables do not store objects directly. They store references (pointers) to objects living on the heap. This is why assignment does not copy objects — it copies references.

```
a = [1, 2, 3]
b = a          # b points to the SAME list on the heap
b.append(4)
print(a)       # [1, 2, 3, 4]  — a is affected too!
```

1.2 Python Memory Model

1.2.1 Mutable vs Immutable Objects

This is one of the most tested Python concepts in interviews — and one of the most misunderstood.

Category	Types	Behaviour on Change
Immutable	int, float, str, tuple, bool, frozenset	Creates a new object

Mutable	list, dict, set, class instances	Modifies in place

```
# Immutable — string
s = "hello"
s += " world"   # creates a NEW string object; original unchanged

# Mutable — list
nums = [1, 2, 3]
nums.append(4)  # modifies the SAME list object in memory
```

⬚ **Trap**	String concatenation inside a loop is O(n²). Always use "".join() — it is O(n).

```
# BAD — O(n²)
result = ""
for char in s:
    result += char   # new string every iteration

# GOOD — O(n)
parts = []
for char in s:
    parts.append(char)
result = "".join(parts)
```

1.2.2 Object Identity vs Equality (is vs ==)

Operator	Checks	Example
==	Value equality	[1,2] == [1,2] → True
is	Same object in memory	[1,2] is [1,2] → False

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)   # True  — same values
print(a is b)   # False — different objects in memory
print(a is c)   # True  — c points to the same object as a

# CPython integer caching (-5 to 256)
x = 100; y = 100
print(x is y)   # True  — cached object

x = 1000; y = 1000
print(x is y)   # False — new objects
```

⬚ **Rule**	Never use 'is' to compare values. Use ==. Reserve 'is' only for: if x is None.

1.2.3 Reference Counting

Every Python object has a counter tracking how many references point to it. When the count reaches zero, the memory is freed immediately.

```
import sys

a = [1, 2, 3]
print(sys.getrefcount(a))  # 2 (a + argument to getrefcount)

b = a
print(sys.getrefcount(a))  # 3 (a + b + argument)

del b
print(sys.getrefcount(a))  # 2 again
```

1.2.4 Garbage Collection Basics

Reference counting alone cannot handle circular references — where two objects reference each other.

```
a = {}
b = {}
a['ref'] = b
b['ref'] = a
# Both have refcount > 0 but are unreachable
# Python's cyclic GC detects and cleans these up
```

Interview	Python uses reference counting as primary memory management. For circular references it cannot handle, Python uses a cyclic garbage collector that runs periodically.

1.2.5 How Python Passes Arguments

Python is neither pass by value nor pass by reference. It is pass by object reference (also called pass by assignment).

```
# Mutable — function modifies the original
def modify_list(lst):
    lst.append(99)
```

```
nums = [1, 2, 3]
modify_list(nums)
print(nums)   # [1, 2, 3, 99]

# Immutable — reassignment doesn't affect original
def modify_int(n):
    n = n + 1      # creates a new local binding

x = 5
modify_int(x)
print(x)   # 5 — unchanged
```

□ **Terms**	Pass by object reference · pass by assignment

1.3 Python Performance Insights

1.3.1 List Internals

A Python list is not a simple array. It is a dynamic array — similar to ArrayList in Java or vector in C++. Python allocates more memory than needed when a list grows (over-allocation), which is why append() is O(1) amortized.

Amortized O(1) means: most appends are instant. Occasionally one append triggers a resize and copy of all elements, but averaged across all appends, the cost is still O(1) per operation.

Operation	Time Complexity	Note
append()	O(1) amortized	Dynamic resize
pop() — from end	O(1)	Fast
pop(0) — from front	O(n)	□ Everything shifts left
insert(i, x)	O(n)	Shifts elements
index(x)	O(n)	Linear scan
in operator	O(n)	Linear scan
Slicing [i:j]	O(k)	k = slice length

□ **Trap**	list.pop(0) is O(n). Use collections.deque when you need O(1) removal from the front.

1.3.2 Dict and Set Hashing

Python dict and set are both implemented as hash tables. When you do d[key], Python calls hash(key), uses the hash to find a bucket, and returns the value — all in O(1) average time.

```
# Only hashable (immutable) objects can be dict keys
d = {}
d[(1, 2)] = 'tuple key'     # valid — tuple is hashable
d[[1, 2]] = 'list key'      # TypeError — list is unhashable

# Membership test: set vs list
nums_list = list(range(10_000_000))
nums_set  = set(range(10_000_000))

999_999 in nums_list  # O(n) — scans entire list
999_999 in nums_set   # O(1) — hash lookup
```

Rule	Always use a set for fast membership testing in interview problems.

1.3.3 String Immutability and the Join Pattern

Strings are immutable. Every concatenation with + creates a brand new string object. In a loop with n iterations this is $O(n^2)$ total work — which causes TLE on large inputs.

```
# BAD — O(n²) total
result = ""
for i in range(n):
    result += str(i)   # new string every iteration

# GOOD — O(n) total
parts = []
for i in range(n):
    parts.append(str(i))
result = "".join(parts)   # single allocation at the end
```

1.3.4 Amortized Complexity in Practice

Amortized complexity means the average cost per operation over a sequence of operations — not the worst case of a single operation.

Classic example: Dynamic array resize. 99 appends cost O(1) each. 1 append triggers resize at O(n). Average across 100 operations = still O(1) per append.

Applies to: list.append() · hash table insertion · stack algorithms where each element is pushed and popped at most once.

Interview	Say: "This is O(n) amortized because each element is processed at most twice — once on entry and once on exit from the stack."

1.4 Python Built-in Tools for Algorithms

These are tools that experienced Python engineers use naturally. Knowing them makes your solutions cleaner, shorter, and faster to write in interviews.

1.4.1 enumerate and zip

```
# enumerate — index + value without manual counter
for i, val in enumerate(nums):
    print(i, val)

for i, val in enumerate(nums, start=1):  # start from 1
    print(i, val)

# zip — combine multiple iterables element by element
names  = ['Alice', 'Bob', 'Charlie']
scores = [90, 85, 92]
for name, score in zip(names, scores):
    print(name, score)

# Transpose a matrix
matrix     = [[1,2,3],[4,5,6],[7,8,9]]
transposed = list(zip(*matrix))
# [(1,4,7), (2,5,8), (3,6,9)]
```

1.4.2 lambda, sorted, min, max with key

```
# lambda creates a small anonymous function
square = lambda x: x * x

# Most useful as a key function in sorting
pairs = [(1,3),(2,1),(3,2)]
pairs.sort(key=lambda x: x[1])    # sort by second element
# [(2,1),(3,2),(1,3)]

words = ['banana','apple','cherry','date']
words.sort(key=lambda w: len(w))   # sort by length
```

```
students = [('Alice',90),('Bob',75),('Charlie',85)]
top = max(students, key=lambda s: s[1])   # ('Alice', 90)

# Sort descending — negate the key
pairs.sort(key=lambda x: -x[1])
```

1.4.3 List Comprehensions and Generator Expressions

List comprehensions are faster than equivalent for loops — they are optimised at the bytecode level.

```
# Basic comprehension
squares = [x * x for x in range(10)]

# With condition
evens = [x for x in range(20) if x % 2 == 0]

# Flatten 2D matrix
matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat = [val for row in matrix for val in row]

# Dictionary comprehension
freq = {char: s.count(char) for char in set(s)}

# Set comprehension
unique_lengths = {len(word) for word in words}

# Generator expression — lazy, memory efficient
total = sum(x * x for x in range(10_000_000))  # no list created
```

1.4.4 defaultdict and Counter

defaultdict automatically creates a default value for missing keys — no more KeyError.

```
from collections import defaultdict

# Frequency counting
freq = defaultdict(int)
for char in 'hello':
    freq[char] += 1

# Grouping — common interview pattern (build adjacency list)
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)
```

Counter counts elements instantly and gives you powerful arithmetic methods.

```
from collections import Counter

s = 'mississippi'
freq = Counter(s)
# Counter({'s':4, 'i':4, 'p':2, 'm':1})

freq.most_common(2)   # [('s',4), ('i',4)]

# Counter arithmetic (very useful for anagram / sliding window)
c1 = Counter('aab')
c2 = Counter('abb')
c1 + c2   # Counter({'a':3,'b':3})
c1 - c2   # Counter({'a':1})
c1 & c2   # intersection: Counter({'a':1,'b':1})
c1 | c2   # union:        Counter({'a':2,'b':2})
```

1.4.5 deque from collections

A deque (double-ended queue) supports O(1) append and pop from both ends. A regular list is O(n) for operations at the front.

```
from collections import deque

q = deque()
q.append(1)       # O(1) — add to right
q.appendleft(0)   # O(1) — add to left
q.pop()           # O(1) — remove from right
q.popleft()       # O(1) — remove from left  (O(n) on a list!)

# maxlen — auto-discards oldest elements
window = deque(maxlen=3)
for num in [1,2,3,4,5]:
    window.append(num)
# deque([3,4,5], maxlen=3)
```

Use for	BFS queue · Sliding window · Monotonic queue

1.4.6 heapq module

Python's heapq implements a min-heap on top of a regular list. The smallest element is always at index 0.

```
import heapq

nums = [5,1,3,2,4]
heapq.heapify(nums)              # O(n) — convert list to heap
heapq.heappush(nums, 0)          # O(log n) — insert
smallest = heapq.heappop(nums)   # O(log n) — remove minimum
peek     = nums[0]               # O(1)     — view minimum

heapq.nlargest(3, nums)   # top 3 largest
heapq.nsmallest(3, nums)  # top 3 smallest

# Max-heap trick — negate values
max_heap = [-n for n in nums]
heapq.heapify(max_heap)
largest = -heapq.heappop(max_heap)

# Priority queue with tuples
heap = []
heapq.heappush(heap, (priority, value))
priority, value = heapq.heappop(heap)
```

1.4.7 bisect module

bisect performs binary search on a sorted list in O(log n) — without writing binary search yourself.

```
import bisect

nums = [1,3,5,7,9]

# bisect_left  — index of first element >= target
bisect.bisect_left(nums, 5)    # 2

# bisect_right — index of first element >  target
bisect.bisect_right(nums, 5)   # 3

# Insert while maintaining sorted order
bisect.insort(nums, 4)         # [1,3,4,5,7,9]

# Example — count elements less than target
count = bisect.bisect_left(nums, 5)   # 2 elements < 5

# bisect_left vs bisect_right on duplicates
nums = [1,3,5,5,5,7]
bisect.bisect_left(nums, 5)    # 2  (leftmost 5)
bisect.bisect_right(nums, 5)   # 5  (one past rightmost 5)
```

1.4.8 itertools

```
import itertools

# combinations — order doesn't matter
list(itertools.combinations([1,2,3], 2))
# [(1,2),(1,3),(2,3)]

# permutations — order matters
list(itertools.permutations([1,2,3], 2))
# [(1,2),(1,3),(2,1),(2,3),(3,1),(3,2)]

# product — cartesian product
list(itertools.product([0,1], repeat=3))  # all 3-bit binary strings

# accumulate — running cumulative sum
list(itertools.accumulate([1,2,3,4,5]))
# [1,3,6,10,15]

# groupby — group consecutive equal elements
for key, group in itertools.groupby('aaabbbcc'):
    print(key, list(group))
# a ['a','a','a']   b ['b','b','b']   c ['c','c']
```

1.4.9 functools — lru_cache and cache

lru_cache is Python's built-in memoization decorator. It automatically caches function results so repeated calls with the same arguments return instantly. This is the fastest way to write top-down dynamic programming in Python.

```
from functools import lru_cache, cache

# lru cache — classic memoization
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

print(fib(100))   # instant — each value computed once

# cache (Python 3.9+) — simpler, no size limit
@cache
def dp(i, j):
    # your 2D DP logic here
    pass

# functools.reduce — cumulative application
from functools import reduce
product = reduce(lambda a, b: a * b, [1,2,3,4,5])  # 120
```

1.5 Common Python Pitfalls in Interviews

These are bugs that experienced developers recognise immediately but beginners fall into repeatedly. Knowing these makes you look sharp in interviews.

1.5.1 Mutable Default Arguments

One of the most famous Python bugs. Default arguments are evaluated once when the function is defined — not each time it is called.

```
# WRONG — the list is created ONCE and shared across all calls
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item(1))   # [1]
print(add_item(2))   # [1, 2]  — unexpected!
print(add_item(3))   # [1, 2, 3] — unexpected!

# CORRECT — use None and create inside the function
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

1.5.2 Shallow vs Deep Copy

```
import copy

original = [[1,2],[3,4]]

# Assignment — same object
ref = original
ref[0].append(99)
print(original)   # [[1,2,99],[3,4]] — modified!

# Shallow copy — new outer list, same inner lists
shallow = original.copy()   # or list(original) or original[:]
shallow[0].append(88)
print(original)   # [[1,2,99,88],[3,4]] — still modified!

# Deep copy — completely independent
deep = copy.deepcopy(original)
deep[0].append(77)
print(original)   # [[1,2,99,88],[3,4]] — NOT modified
```

Rule	When copying a 2D grid or nested structure in DP or backtracking, use copy.deepcopy() or reconstruct manually.

1.5.3 Integer Overflow? Python Does Not Have It

In C++ and Java, integers overflow when they exceed the maximum value for their type (e.g. 2^31 - 1 for 32-bit int). Python integers have arbitrary precision — they never overflow. Python automatically allocates more memory as numbers grow.

```
print(2 ** 1000)   # works perfectly — a huge number, no overflow
```

Interview	Mention this to interviewers when a problem involves large numbers. It is a real advantage of Python.

1.5.4 Floating Point Comparison Issues

```
print(0.1 + 0.2 == 0.3)   # False!
print(0.1 + 0.2)           # 0.30000000000000004

# Safe comparison
import math
math.isclose(0.1 + 0.2, 0.3)      # True
abs(0.1 + 0.2 - 0.3) < 1e-9       # True
```

Note	This is not a Python bug — it is how IEEE 754 floating point works in every language including C, Java, and JavaScript.

1.5.5 List Multiplication Trap — 2D Grid Initialisation

This is one of the most common bugs in 2D array initialisation in coding interviews.

```
# WRONG — n references to the SAME inner list
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)
# [[1,0,0],[1,0,0],[1,0,0]]  — all rows changed!

# CORRECT — independent inner lists
grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 1
```

```
print(grid)
# [[1,0,0],[0,0,0],[0,0,0]]  — only row 0 changed
```

Rule	Always initialise 2D grids with list comprehension. Never use * multiplication for nested lists.

Section 1 — Quick Revision Cheat Sheet

Topic	Key Fact
Python Execution	source → bytecode → PVM (CPython)
Call Stack	each function call = one stack frame, limit = 1000
Stack vs Heap	variables on stack, objects on heap
Mutable	list, dict, set — changed in place
Immutable	int, str, tuple — new object on change
Argument passing	pass by object reference
list.append()	O(1) amortized — dynamic array
dict/set lookup	O(1) average — hash table
String concat loop	use "".join() not +=
lru_cache / cache	instant memoization for top-down DP
deque	O(1) both ends — use for BFS and sliding window
heapq	min-heap by default, negate for max-heap
bisect	binary search on sorted list in O(log n)
2D grid init	[[0]*n for _ in range(m)] — NOT [[0]*n]*m
Integer overflow	does not exist in Python — arbitrary precision
is vs ==	use == for values, is only for None checks

End of Section 1. Continue with Section 2 — Data Structures Cheat Sheet.